In [111…

```python
# CARGA DE DATOS #

import pandas as pd
# test comment
# Ruta al archivo de Excel
archivo_excel = './IIYNT-REQ-BD-003.xlsx' #../

# Número de la fila desde la cual querés empezar (0-indexado)
fila_inicio = 3

# Leer el archivo Excel
init_df = pd.read_excel(archivo_excel)

# Mostrar las primeras filas del DataFrame para confirmar que se leyó bien
init_df.head()

# TRANSFORMACIÓN DE DATOS #

desired_columns = [
    'Temperatura Ambiental (°C)',
    'Humedad',
    'Temperatura de la muestra (°C)',
    'pH',
    'CE\n(µS/cm)',
    'STD\n(mg/L)',
    'STS\n(mL sed/L)',
    'OD\n(mg/L)',
    'Nivel (cm)',
    'Turbidez (NTU)',
    'Dureza\n(mg CaCO3/L)',
    'Cloruros\n(mg Cl-/L)'
]
df = init_df.iloc[2:123][desired_columns]
df
```

Out[111…

| | Temperatura Ambiental (°C) | Humedad | Temperatura de la muestra (°C) | pH | CE\n(µS/cm) | STD\n(mg/L) | STS\n(mL sed/L) | OD\n(mg/L) |
|---|---|---|---|---|---|---|---|---|
| **2** | 16 | 0.929 | 18 | 8.2 | 1240 | 610 | NaN | 5.37 |
| **3** | 17 | 0.47 | 19 | 8.3 | 1630 | 810 | 1.8 | 4.3 |
| **4** | 11.9 | 0.47 | 13 | 8.1 | 1000 | 490 | 18 | 5.3 |
| **5** | 11.9 | 0.47 | 13 | 8.2 | 1000 | 490 | 18 | 4.67 |
| **6** | 11.9 | 0.47 | 13 | 8.3 | 1350 | 670 | 0.1 | 7.01 |
| **…** | … | … | … | … | … | … | … | … |
| **118** | 12.8 | 0.57 | 15.3 | 8 | 1110 | 550 | 18 | 4.18 |
| **119** | 12.8 | 0.57 | 15.3 | 8.1 | 1080 | 550 | 18 | 4.23 |
| **120** | 12.8 | 0.57 | 15.4 | 8.4 | 1050 | 520 | 44 | 1.96 |
| **121** | 12.8 | 0.57 | 15.4 | 8.4 | 1070 | 530 | 44 | 1.67 |
| **122** | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN |

121 rows × 12 columns

In [112…

```python
#ver los que tienen valores nulos o no asignados

#df.isnull().sum()
df.isna().sum()
```

```python
#Eliminar las filas que tienen valores nulos o no asignados
#df.dropna(inplace=True) #aca se modifica el Dataframe original con inplace=true

#Si asigno a nueva variable no hace falta inplace=true pero no modifica el Dataframe
#df_sin_nulos=df.dropna()

#Como opcion puedo llenar los valores nulos con datos adecuados:
#fillna(method='bfill') (llenar con el valor siguiente de la columna)
#fillna(method='ffill') (llenar con el valor anterior de la columna)
#fillna(0) (llenar con ceros)

mediaNivel=df['Nivel (cm)'].mean()
df['Nivel (cm)']=df['Nivel (cm)'].fillna(mediaNivel)

mediaSTS=df['STS\n(mL sed/L)'].mean()
df['STS\n(mL sed/L)']=df['STS\n(mL sed/L)'].fillna(mediaSTS)

mediaTurbidez=df['Turbidez (NTU)'].mean()
df['Turbidez (NTU)']=df['Turbidez (NTU)'].fillna(mediaTurbidez)

mediaDureza=df['Dureza\n(mg CaCO3/L)'].mean()
df['Dureza\n(mg CaCO3/L)']=df['Dureza\n(mg CaCO3/L)'].fillna(mediaDureza)

mediaCloruros=df['Cloruros\n(mg Cl-/L)'].mean()
df['Cloruros\n(mg Cl-/L)']=df['Cloruros\n(mg Cl-/L)'].fillna(mediaCloruros)

df_sin_nulos=df.copy()
df_sin_nulos.isna().sum()

#Eliminar las filas que tienen valores nulos o no asignados
df_sin_nulos.dropna(inplace=True) #aca se modifica el Dataframe original con inplace=
df_sin_nulos.isna().sum()
```

Out[112…

```
Temperatura Ambiental (°C)        0
Humedad                           0
Temperatura de la muestra (°C)    0
pH                                0
CE\n(µS/cm)                       0
STD\n(mg/L)                       0
STS\n(mL sed/L)                   0
OD\n(mg/L)                        0
Nivel (cm)                        0
Turbidez (NTU)                    0
Dureza\n(mg CaCO3/L)              0
Cloruros\n(mg Cl-/L)              0
dtype: int64
```

In [113…

```python
df_sin_nulos.describe()
```

Out[113…

| | STS\n(mL sed/L) | Nivel (cm) | Turbidez (NTU) | Dureza\n(mg CaCO3/L) | Cloruros\n(mg Cl-/L) |
|---|---|---|---|---|---|
| **count** | 120.000000 | 120.000000 | 120.000000 | 120.000000 | 120.000000 |
| **mean** | 68.326496 | 39.937500 | 155.879746 | 172.495798 | 96.871795 |
| **std** | 109.747232 | 10.695239 | 245.077765 | 54.242666 | 27.726012 |
| **min** | 0.100000 | 20.000000 | 2.680000 | 86.000000 | 15.000000 |
| **25%** | 28.000000 | 30.000000 | 38.800000 | 135.000000 | 74.500000 |
| **50%** | 50.000000 | 39.937500 | 79.650000 | 157.000000 | 104.000000 |
| **75%** | 70.000000 | 45.000000 | 148.500000 | 204.000000 | 118.000000 |
| **max** | 650.000000 | 70.000000 | 1000.000000 | 316.000000 | 156.000000 |

In [114…

```python
def min_max_info(dataframe: pd.DataFrame) -> pd.DataFrame:
    name = list()
    minimums = list()
    maximums = list()
    median = list()
    std = list()
    for i, key in enumerate(dataframe.columns):
        name.append(key)
        minimums.append(dataframe[key].min())
        maximums.append(dataframe[key].max())
        median.append(dataframe[key].median())
        std.append(dataframe[key].std())

    return pd.DataFrame(
        {
            'name': name,
            'mins': minimums,
            'maxs': maximums,
            'median': median,
            'std': std
        }
    )
min_max_info(df_sin_nulos)
```

Out[114…

| | name | mins | maxs | median | std |
|---|---|---|---|---|---|
| 0 | Temperatura Ambiental (°C) | 10.40 | 26.000 | 14.4000 | 4.380318 |
| 1 | Humedad | 0.36 | 0.929 | 0.5600 | 0.164879 |
| 2 | Temperatura de la muestra (°C) | 12.80 | 28.100 | 16.9000 | 2.915339 |
| 3 | pH | 7.20 | 8.700 | 8.2000 | 0.245005 |
| 4 | CE\n(µS/cm) | 200.00 | 1690.000 | 1290.0000 | 253.967054 |
| 5 | STD\n(mg/L) | 140.00 | 840.000 | 635.0000 | 125.125119 |
| 6 | STS\n(mL sed/L) | 0.10 | 650.000 | 50.0000 | 109.747232 |
| 7 | OD\n(mg/L) | 0.00 | 9.120 | 3.1500 | 2.097971 |
| 8 | Nivel (cm) | 20.00 | 70.000 | 39.9375 | 10.695239 |
| 9 | Turbidez (NTU) | 2.68 | 1000.000 | 79.6500 | 245.077765 |
| 10 | Dureza\n(mg CaCO3/L) | 86.00 | 316.000 | 157.0000 | 54.242666 |
| 11 | Cloruros\n(mg Cl-/L) | 15.00 | 156.000 | 104.0000 | 27.726012 |

In [115…

```python
def normalize(dataframe: pd.DataFrame, min: float, max: float) -> pd.DataFrame:
    return (dataframe - min) / (max - min)

def denormalize(dataframe: pd.DataFrame, min: float, max: float) -> pd.DataFrame:
    return dataframe * (max - min) + min


def normalizer(dataframe: pd.DataFrame) -> pd.DataFrame:
    df = dataframe.copy()
    df['Temperatura Ambiental (°C)']                = normalize(df['Temperatura Ambie
    df['Humedad'] = df['Humedad']
    df['Temperatura de la muestra (°C)']            = normalize(df['Temperatura de la
    df['pH']                                        = normalize(df['pH'],
    df['CE\n(µS/cm)']                               = normalize(df['CE\n(µS/cm)'],
    df['STD\n(mg/L)']                               = normalize(df['STD\n(mg/L)'],
    df['STS\n(mL sed/L)']                           = normalize(df['STS\n(mL sed/L)']
    df['OD\n(mg/L)']                                = normalize(df['OD\n(mg/L)'],
    df['Nivel (cm)']                                = normalize(df['Nivel (cm)'],
    df['Turbidez (NTU)']                            = normalize(df['Turbidez (NTU)'],
    df['Dureza\n(mg CaCO3/L)']                      = normalize(df['Dureza\n(mg CaCO3
```

```
    df['Cloruros\n(mg Cl-/L)']                          = normalize(df['Cloruros\n(mg Cl-
    return df

def denormalizer(dataframe: pd.DataFrame) -> pd.DataFrame:
    df = dataframe.copy()
    df['Temperatura Ambiental (°C)']                    = denormalize(df['Temperatura Amb
    df['Humedad'] = df['Humedad']
    df['Temperatura de la muestra (°C)']                = denormalize(df['Temperatura de
    df['pH']                                            = denormalize(df['pH'],
    df['CE\n(µS/cm)']                                   = denormalize(df['CE\n(µS/cm)'],
    df['STD\n(mg/L)']                                   = denormalize(df['STD\n(mg/L)'],
    df['STS\n(mL sed/L)']                               = denormalize(df['STS\n(mL sed/L)
    df['OD\n(mg/L)']                                    = denormalize(df['OD\n(mg/L)'],
    df['Nivel (cm)']                                    = denormalize(df['Nivel (cm)'],
    df['Turbidez (NTU)']                                = denormalize(df['Turbidez (NTU)'
    df['Dureza\n(mg CaCO3/L)']                          = denormalize(df['Dureza\n(mg CaC
    df['Cloruros\n(mg Cl-/L)']                          = denormalize(df['Cloruros\n(mg C
    return df
```

## Autoencoder basico

In [116…

```
from keras.layers import Input, Dense
from keras.activations import sigmoid, tanh
from keras.models import Model
import numpy as np
from tensorflow.keras.callbacks import TensorBoard
import shutil
import os as os

try:
    shutil.rmtree('./.logs')
except FileNotFoundError:
    pass
os.mkdir('./.logs')
vector_size = len(df_sin_nulos.columns)
dataset = np.array(
    normalizer(df_sin_nulos).astype(np.float16)
)
encoded_size = 2

input_layer = Input(shape=(vector_size, ))

x = Dense(2, activation=sigmoid)(input_layer)
x = Dense(2, activation=sigmoid)(x)

encoded = Dense(encoded_size, activation=sigmoid)(x)
x = Dense(2, activation=sigmoid)(encoded)
x = Dense(2, activation=sigmoid)(x)
output = Dense(vector_size, activation=sigmoid)(x)

autoencoder = Model(input_layer, output) # creo un modelo a partir de lo definido arr
autoencoder.compile(optimizer='adam', loss='mse') # se compila el modelo
autoencoder.summary() # se imprime un resumen
```

```
Model: "model_29"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_26 (InputLayer)       [(None, 12)]              0

 dense_136 (Dense)           (None, 2)                 26

 dense_137 (Dense)           (None, 2)                 6

 dense_138 (Dense)           (None, 2)                 6

 dense_139 (Dense)           (None, 2)                 6

 dense_140 (Dense)           (None, 2)                 6

 dense_141 (Dense)           (None, 12)                36

=================================================================
Total params: 86 (344.00 Byte)
Trainable params: 86 (344.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

# Autoencoder variacional: Un valor entre otros dos valores es correctamente interpolado

## El dataset creado a partir del espacio latente generara valores coherentes.

In [117…

```
from keras.layers import Input, Dense, Lambda
from keras.models import Model
from keras import backend as K
from keras.losses import mean_squared_error
import tensorflow as tf

# Definición de la capa de muestreo
def sampling(args):
    z_mean, z_log_var = args
    batch = K.shape(z_mean)[0]
    dim = K.int_shape(z_mean)[1]
    epsilon = K.random_normal(shape=(batch, dim))
    return z_mean + K.exp(0.5 * z_log_var) * epsilon

# Supongamos que vector_size y encoded_size ya están definidos
# vector_size = ...
# encoded_size = ...

input_layer = Input(shape=(vector_size,))
encoder_size = 2
# Encoder
x = Dense(encoder_size, activation=sigmoid)(input_layer)
x = Dense(encoder_size, activation=sigmoid)(x)

z_mean = Dense(encoder_size)(x)
z_log_var = Dense(encoder_size)(x)

# Capa de muestreo
z = Lambda(sampling, output_shape=(encoder_size,))([z_mean, z_log_var])

# Decoder
decoder_h1 = Dense(encoder_size, activation=sigmoid)
decoder_h2 = Dense(encoder_size, activation=sigmoid)
```

```
decoder_output = Dense(vector_size, activation=sigmoid)

x_decoded = decoder_h1(z)
x_decoded = decoder_h2(x_decoded)
x_decoded = decoder_output(x_decoded)

# Modelo completo
vae = Model(input_layer, x_decoded)

# Función de pérdida personalizada
xent_loss = vector_size * mean_squared_error(input_layer, x_decoded)
kl_loss = - 0.5 * K.sum(1 + z_log_var - K.square(z_mean) - K.exp(z_log_var), axis=-1)
vae_loss = K.mean(xent_loss + kl_loss)

vae.add_loss(vae_loss)
vae.compile(optimizer='adam')

vae.summary()
```

```
Model: "model_30"
__________________________________________________________________________________________________
 Layer (type)                   Output Shape                 Param #   Connected to
==================================================================================================
 input_27 (InputLayer)          [(None, 12)]                 0         []

 dense_142 (Dense)              (None, 2)                    26        ['input_27[0][0]']

 dense_143 (Dense)              (None, 2)                    6         ['dense_142[0]
[0]']

 dense_144 (Dense)              (None, 2)                    6         ['dense_143[0]
[0]']

 dense_145 (Dense)              (None, 2)                    6         ['dense_143[0]
[0]']

 lambda_10 (Lambda)             (None, 2)                    0         ['dense_144[0]
[0]',
                                                                         'dense_145[0]
[0]']

 dense_146 (Dense)              (None, 2)                    6         ['lambda_10[0]
[0]']

 dense_147 (Dense)              (None, 2)                    6         ['dense_146[0]
[0]']

 dense_148 (Dense)              (None, 12)                   36        ['dense_147[0]
[0]']

 tf.__operators__.add_20 (T     (None, 2)                    0         ['dense_145[0]
[0]']
 FOpLambda)

 tf.math.square_10 (TFOpLam     (None, 2)                    0         ['dense_144[0]
[0]']
 bda)

 tf.convert_to_tensor_10 (T     (None, 12)                   0         ['dense_148[0]
[0]']
 FOpLambda)

 tf.cast_10 (TFOpLambda)        (None, 12)                   0         ['input_27[0][0]']

 tf.math.subtract_20 (TFOpL     (None, 2)                    0         ['tf.__operators_
_.add_20[0][0
 ambda)                                                                  ]',
                                                                         'tf.math.square_1
0[0][0]']

 tf.math.exp_10 (TFOpLambda     (None, 2)                    0         ['dense_145[0]
[0]']
 )

 tf.math.squared_difference     (None, 12)                   0         ['tf.convert_to_te
nsor_10[0][0
 _10 (TFOpLambda)                                                        ]',
                                                                         'tf.cast_10[0]
[0]']

 tf.math.subtract_21 (TFOpL     (None, 2)                    0         ['tf.math.subtract
_20[0][0]',
 ambda)                                                                  'tf.math.exp_10
```

```
[0][0]']

 tf.math.reduce_mean_20 (TF  (None,)                      0         ['tf.math.squared_
difference_1
 OpLambda)                                                          0[0][0]']

 tf.math.reduce_sum_10 (TFO  (None,)                      0         ['tf.math.subtract
_21[0][0]']
 pLambda)

 tf.math.multiply_20 (TFOpL  (None,)                      0         ['tf.math.reduce_m
ean_20[0][0]
 ambda)                                                             ']

 tf.math.multiply_21 (TFOpL  (None,)                      0         ['tf.math.reduce_s
um_10[0][0]'
 ambda)                                                             ]

 tf.__operators__.add_21 (T  (None,)                      0         ['tf.math.multiply
_20[0][0]',
 FOpLambda)                                                          'tf.math.multiply
_21[0][0]']

 tf.math.reduce_mean_21 (TF  ()                           0         ['tf.__operators_
_.add_21[0][0
 OpLambda)                                                          ]']

 add_loss_10 (AddLoss)       ()                           0         ['tf.math.reduce_m
ean_21[0][0]
                                                                    ']

==================================================================================
============
Total params: 92 (368.00 Byte)
Trainable params: 92 (368.00 Byte)
Non-trainable params: 0 (0.00 Byte)
__________________________________________________________________________________
____________
```

In [118…

```python
tensorboard = TensorBoard(log_dir='./.logs') #creo instancia para guardar
epochs = 20000 # cantidad de veces a entranar o pasos para realizar el descenso del g
autoencoder.fit(
    x=dataset,
    y=dataset,
    epochs=epochs,
    batch_size=len(dataset), #tamaño del bloque de entrenamiento
    shuffle=True,
    verbose=0, # que no envie salidas
    callbacks=[tensorboard]

)
```

Out[118…

```
<keras.src.callbacks.History at 0x7f8017f32dc0>
```

In [119…

```python
generator = Model(encoded, output) #va del vector de clasificacion hasta la salida
generator.summary()
```

```
Model: "model_31"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_28 (InputLayer)       [(None, 2)]               0

 dense_139 (Dense)           (None, 2)                 6

 dense_140 (Dense)           (None, 2)                 6

 dense_141 (Dense)           (None, 12)                36

=================================================================
Total params: 48 (192.00 Byte)
Trainable params: 48 (192.00 Byte)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

In [120…

```
encoder = Model(input_layer, z) #encoded para el basico #va desde el input a los dato
encoder.summary()
```

```
Model: "model_32"
__________________________________________________________________________________________________
 Layer (type)                Output Shape                 Param #   Connected to
==================================================================================================
 input_27 (InputLayer)       [(None, 12)]                 0         []

 dense_142 (Dense)           (None, 2)                    26        ['input_27[0][0]']

 dense_143 (Dense)           (None, 2)                    6         ['dense_142[0]
[0]']

 dense_144 (Dense)           (None, 2)                    6         ['dense_143[0]
[0]']

 dense_145 (Dense)           (None, 2)                    6         ['dense_143[0]
[0]']

 lambda_10 (Lambda)          (None, 2)                    0         ['dense_144[0]
[0]',
                                                                       'dense_145[0]
[0]']

==================================================================================================
Total params: 44 (176.00 Byte)
Trainable params: 44 (176.00 Byte)
Non-trainable params: 0 (0.00 Byte)
__________________________________________________________________________________________________
```

In [121…

```
amount = 10000 #cantidad de predicciones
generated = np.zeros(shape=(amount, vector_size))
params = np.random.rand(amount, encoded_size)
generated = generator.predict(params)
gen_df = pd.DataFrame(generated)
gen_df.columns = df_sin_nulos.columns
gen_df = denormalizer(gen_df)
gen_df.to_csv('./dataset_simulado_2_neuronas_variacional.csv')
gen_df
```

```
313/313 [==============================] - 0s 459us/step
```

Out[121…

| | Temperatura Ambiental (°C) | Humedad | Temperatura de la muestra (°C) | pH | CE\n(µS/cm) | STD\n(mg/L) | STS\n(mL sed/L) | OD\r |
|---|---|---|---|---|---|---|---|---|
| **0** | 22.803240 | 0.605684 | 19.450586 | 7.412695 | 1101.511475 | 542.292603 | 397.854553 | 0. |
| **1** | 23.955772 | 0.624442 | 19.739929 | 7.416586 | 1488.096313 | 738.403870 | 587.675720 | 0. |
| **2** | 18.992764 | 0.598855 | 18.143635 | 7.855973 | 1254.639038 | 618.386475 | 128.867798 | 0. |
| **3** | 23.877537 | 0.621691 | 19.730534 | 7.404345 | 1430.373901 | 708.633179 | 568.188293 | 0. |
| **4** | 14.744596 | 0.573515 | 16.787590 | 8.230697 | 955.794250 | 470.052246 | 19.233513 | 5. |
| **...** | ... | ... | ... | ... | ... | ... | ... | |
| **9995** | 24.037455 | 0.627723 | 19.746811 | 7.432834 | 1552.921997 | 771.972168 | 609.589722 | 0. |
| **9996** | 14.435667 | 0.560298 | 16.759583 | 8.164203 | 651.871460 | 327.780914 | 13.594338 | 5. |
| **9997** | 14.059135 | 0.541483 | 16.740902 | 8.061472 | 350.365540 | 196.823318 | 8.483684 | 5. |
| **9998** | 22.828659 | 0.605055 | 19.464161 | 7.404463 | 1081.603760 | 532.464966 | 397.350159 | 0. |
| **9999** | 15.831663 | 0.592339 | 17.060848 | 8.235927 | 1357.435059 | 670.271790 | 39.793663 | 3. |

10000 rows × 12 columns

In [122…

```
min_max_info(df_sin_nulos)
```

Out[122…

| | name | mins | maxs | median | std |
|---|---|---|---|---|---|
| **0** | Temperatura Ambiental (°C) | 10.40 | 26.000 | 14.4000 | 4.380318 |
| **1** | Humedad | 0.36 | 0.929 | 0.5600 | 0.164879 |
| **2** | Temperatura de la muestra (°C) | 12.80 | 28.100 | 16.9000 | 2.915339 |
| **3** | pH | 7.20 | 8.700 | 8.2000 | 0.245005 |
| **4** | CE\n(µS/cm) | 200.00 | 1690.000 | 1290.0000 | 253.967054 |
| **5** | STD\n(mg/L) | 140.00 | 840.000 | 635.0000 | 125.125119 |
| **6** | STS\n(mL sed/L) | 0.10 | 650.000 | 50.0000 | 109.747232 |
| **7** | OD\n(mg/L) | 0.00 | 9.120 | 3.1500 | 2.097971 |
| **8** | Nivel (cm) | 20.00 | 70.000 | 39.9375 | 10.695239 |
| **9** | Turbidez (NTU) | 2.68 | 1000.000 | 79.6500 | 245.077765 |
| **10** | Dureza\n(mg CaCO3/L) | 86.00 | 316.000 | 157.0000 | 54.242666 |
| **11** | Cloruros\n(mg Cl-/L) | 15.00 | 156.000 | 104.0000 | 27.726012 |

In [123…

```
min_max_info(gen_df)
```

Out[123…

| | name | mins | maxs | median | std |
|---|---|---|---|---|---|
| 0 | Temperatura Ambiental (°C) | 14.043187 | 24.091227 | 15.991282 | 4.209682 |
| 1 | Humedad | 0.540661 | 0.630008 | 0.593533 | 0.029286 |
| 2 | Temperatura de la muestra (°C) | 16.740215 | 19.750435 | 17.110497 | 1.328493 |
| 3 | pH | 7.375314 | 8.272366 | 8.068447 | 0.354144 |
| 4 | CE\n(µS/cm) | 341.114258 | 1594.900635 | 1138.191162 | 408.071625 |
| 5 | STD\n(mg/L) | 192.999695 | 793.765564 | 560.006897 | 195.386292 |
| 6 | STS\n(mL sed/L) | 8.312063 | 624.362244 | 43.159931 | 242.922287 |
| 7 | OD\n(mg/L) | 0.001306 | 5.638665 | 2.833921 | 2.572573 |
| 8 | Nivel (cm) | 24.377001 | 43.774666 | 31.476284 | 5.320664 |
| 9 | Turbidez (NTU) | 40.683105 | 995.500854 | 87.264290 | 450.327423 |
| 10 | Dureza\n(mg CaCO3/L) | 142.177155 | 178.435730 | 155.308075 | 10.814542 |
| 11 | Cloruros\n(mg Cl-/L) | 15.019542 | 132.087982 | 78.844452 | 37.408890 |

In [124…

```
colors = encoder.predict(dataset)
color_max = colors.max()
color_min = colors.min()

colored_dataset = df_sin_nulos.copy() #copio el dataset original y lo coloreo con el
#print(colors.shape)


def color_row(row, color_vector):
    #print(color_vector)
    color_vector_normalized = (color_vector - color_min)/(color_max - color_min)
    color_rgb = tuple((color_vector_normalized*255).astype(np.int16))
    color_hex = '#{:02X}{:02X}{:02X}'.format(color_rgb[0], color_rgb[1],255)
    return [f'background-color: {color_hex};' for _ in row.index]

# Create an iterator for the colors
color_iterator = iter(colors)

# Apply colors to each row
styled_df = colored_dataset.style.apply(lambda row: color_row(row, next(color_iterato
styled_df.to_excel("colored_dataset_variacional.xlsx", engine='openpyxl')
```

```
4/4 [==============================] - 0s 2ms/step
```

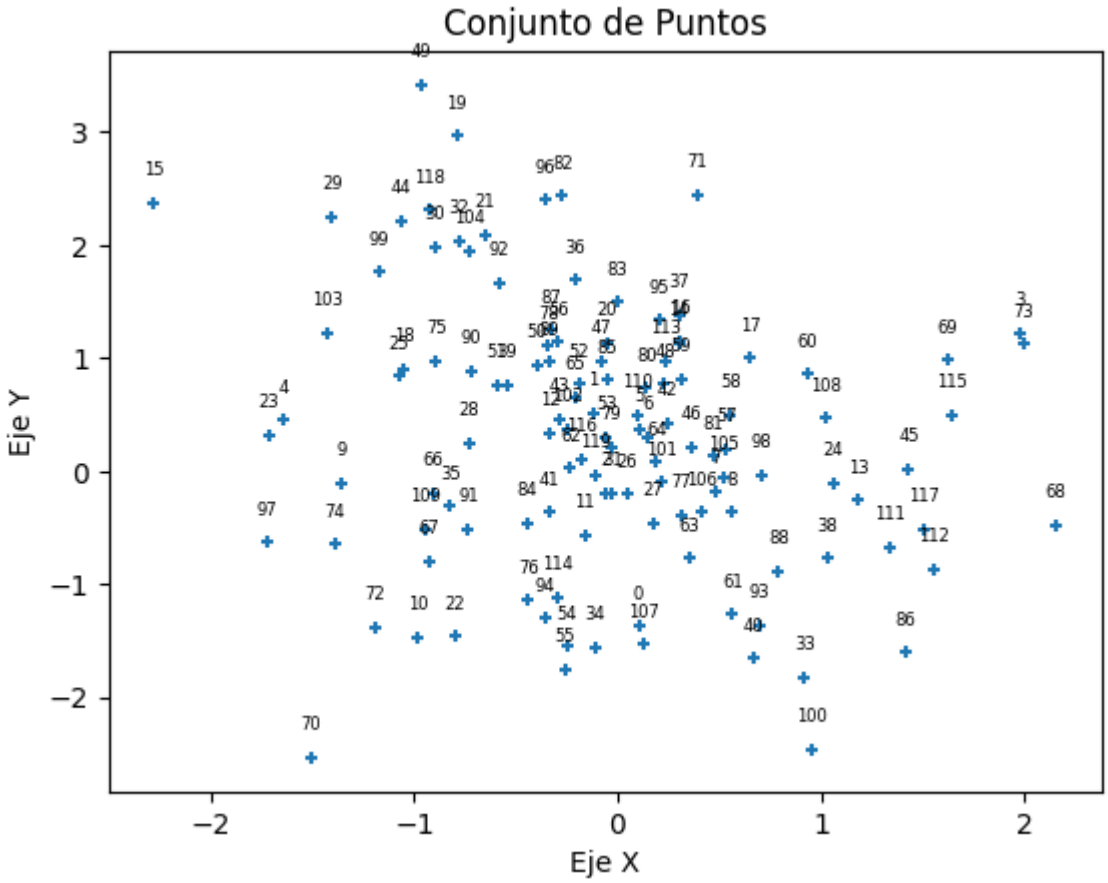
In [125…

```
import matplotlib.pyplot as plt


# Dibujamos los puntos
plt.scatter(colors.T[0], colors.T[1], s=20, marker='+')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')
plt.title('Conjunto de Puntos')

# Añadimos índices a cada punto
for i, (xi, yi) in enumerate(colors):
    plt.annotate(str(i), (xi, yi), textcoords="offset points", xytext=(0,10), ha='cen

plt.show()
```




## Justificación Matemática

Un Variational Autoencoder (VAE) no solo aprende a mapear los datos a un espacio latente y a reconstruirlos, sino que también trata de hacer que la distribución de esos puntos en el espacio latente siga una distribución normal multivariada, típicamente con media cero y varianza unitaria.

### ¿Por qué es válido muestrear cerca de la media (0,0) con una cierta desviación estándar?

1. **Distribución Aproximada**: El VAE intenta que la distribución latente ( $q(z|x)$ ) se aproxime a una distribución normal estándar ( $p(z)$ ), donde ( $z$ ) es la variable latente y ( $x$ ) son tus datos. Esto se logra mediante la divergencia KL en la función de pérdida. La Divergencia Kullback-Leibler (KL) es una medida de cuán diferente es una distribución de probabilidad PP con respecto a otra distribución QQ.

2. **Densidad de Probabilidad**: En una distribución normal, la máxima densidad de probabilidad está en la media. Esto significa que los puntos muestreados cerca de la media son, en teoría, "más probables" o "más representativos" del conjunto de datos original.

3. **Varianza Controlada**: Al añadirse un vector de desviación estándar, se esta controlando cuánto nos alejamos de la media. Esto permite muestrear puntos que son plausibles pero que también introducen variabilidad.

### Cautelas

1. **Interpolación vs Extrapolación**: Muestrear dentro de la "nube" de puntos latentes es generalmente seguro (interpolación). Pero si te alejás demasiado, podés entrar en el terreno de la extrapolación, que es más riesgoso y podría generar puntos no representativos.

2. **Validación**: Sería muy útil validar los puntos generados con expertos en química para asegurarte de que son plausibles.

3. **Estadísticas Descriptivas**: Además, podrían se estadísticas descriptivas y hacer comparaciones gráficas (e.g., histogramas, boxplots) entre los datos originales y los generados para evaluar cuán bien los datos generados representan al conjunto original.